

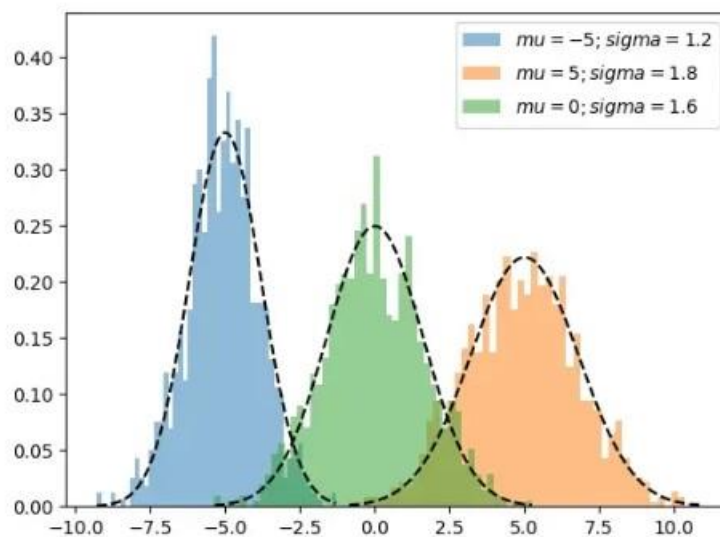
Clustering in Machine Learning

Refer note in Module 2

Gaussian Mixture Model

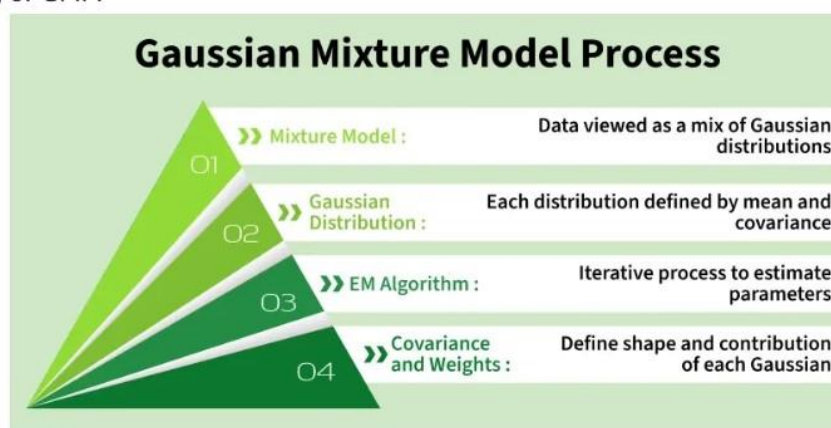
Gaussian Mixture Model (GMM) is a probabilistic clustering technique that models data as a combination of multiple Gaussian distributions, allowing more flexible grouping of data points.

- Assigns each data point a probability of belonging to different clusters
- Can handle overlapping clusters effectively
- Uses mean and covariance to define cluster shape



The above shown graph shows a three one-dimensional Gaussian distributions with distinct means and variances. Each curve represents the theoretical probability density function (PDF) of a normal distribution, highlighting differences in location and spread.

Working of GMM



Working of Gaussian Mixture Model

A Gaussian Mixture Model assumes that the data is generated from a mixture of K Gaussian distributions, each representing a cluster. Every Gaussian has its own mean μ_k , covariance Σ_k and mixing weight π_k .

1. Posterior Probability (Cluster Responsibility)

For a given data point x_n , the probability that it belongs to cluster k :

$$P(z_n = k \mid x_n) = \frac{\pi_k \cdot \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \cdot \mathcal{N}(x_n \mid \mu_j, \Sigma_j)}$$

where:

- z_n is a latent variable indicating cluster assignment
- π_k is the mixing probability of the k -th Gaussian.
- $\mathcal{N}(x_n \mid \mu_k, \Sigma_k)$ is the Gaussian distribution with mean μ_k and covariance Σ_k

2. Likelihood of a Data Point

The total likelihood of observing x_n under all Gaussians is:

$$P(x_n) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}(x_n \mid \mu_k, \Sigma_k)$$

This represents how well the mixture as a whole explains the data point.

3. Expectation-Maximization (EM) Algorithm

GMM parameters are estimated using the EM algorithm:

E-step (Expectation): Compute the responsibility of each cluster for every data point using current parameter values.

M-step (Maximization): Update

- Means μ_k
- Covariances Σ_k
- Mixing coefficients using the responsibilities from the E-step. The process continues until the model's log-likelihood stabilizes.

4. Log-Likelihood of the Mixture Model

The objective optimized by EM is:

$$L(\mu, \Sigma, \pi) = \prod_{n=1}^N \sum_{k=1}^K \pi_k \cdot \mathcal{N}(x_n \mid \mu_k, \Sigma_k)$$

EM increases this likelihood in every iteration.

Cluster Shapes in GMM

In GMM, each cluster is a Gaussian defined by:

- **Mean (μ):** Center of the cluster.
- **Covariance (Σ):** Controls the shape, orientation and spread of the cluster.

Because covariance matrices allow elliptical shapes, GMM can model:

- elongated clusters
- tilted clusters
- overlapping clusters

Because covariance matrices allow elliptical shapes, GMM can model:

- elongated clusters
- tilted clusters
- overlapping clusters

This makes GMM more flexible than methods like K-Means, which assumes only spherical clusters.

Visualizing GMM often involves:

- Scatter plots showing raw data
- Elliptical contours (or KDE curves) showing the shape of each Gaussian component

These illustrate how GMM adapts to complex, real-world data distributions.

Use-Cases

- **Clustering:** Discover underlying groups or structure in data (marketing, medicine, genetics).
- **Anomaly Detection:** Identify outliers or rare events (fraud, medical errors).
- **Image Segmentation:** Separate images into meaningful regions (medical, remote sensing).
- **Density Estimation:** Model complex probability distributions for generative modeling.

Advantages

- **Flexible Cluster Shapes:** Models ellipsoidal and overlapping clusters.
- **Soft Assignments:** Assigns probabilistic cluster membership instead of hard labels.
- **Handles Missing Data:** Robust to incomplete observations.
- **Interpretable Parameters:** Each Gaussian's mean, covariance and weight are easy to interpret.

Limitations

- **Initialization Sensitive:** Results depend on starting parameter values can get stuck in local optima.
- **Computation Intensive:** Slow for high-dimensional or very large datasets.
- **Assumes Gaussian Distributions:** Not suitable for non-Gaussian cluster shapes.
- **Requires Cluster Number:** Must specify the number of components/clusters before fitting.

K means Clustering – Introduction

K-Means Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- Helps identify natural groupings in unlabeled datasets
- Works by grouping points based on distance to cluster centers
- Commonly used in customer segmentation, image compression and pattern discovery
- Useful when you need structure from raw, unorganized data

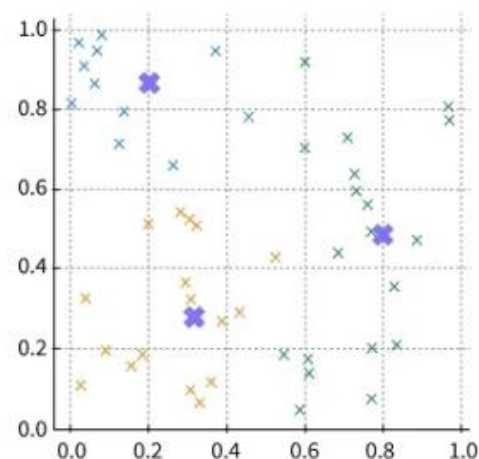
Working of K-Means Clustering

Suppose we are given a data set of items with certain features and values for these features like a vector. The task is to categorize those items into groups. To achieve this we will use the K-means algorithm. "k" represents the number of groups or clusters we want to classify our items into.

Repeat Until Convergence

This process repeats until the centroids stabilize and do not move further.

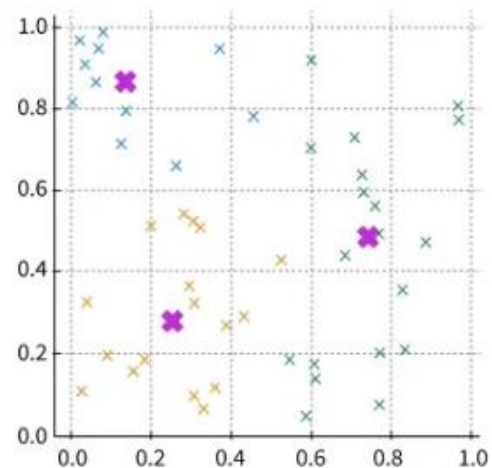
- ✳ Final Centroids
- ✕ Cluster 1
- ✕ Cluster 2
- ✕ Cluster 3



Update Centroids

Centroids are recalculated as the mean of the points in each cluster

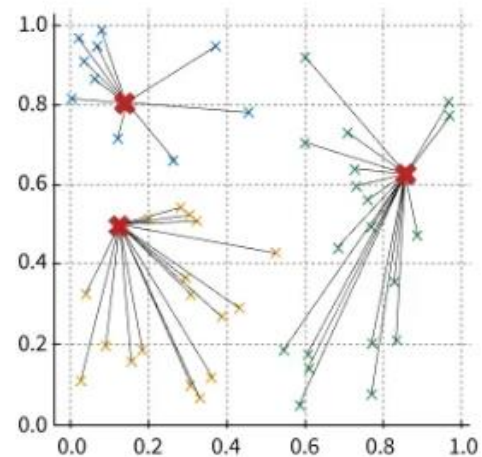
- ✱ New Centroids
- ✕ Cluster 1
- ✕ Cluster 2
- ✕ Cluster 3



Assign Points to Nearest Centroid

Each point is assigned to the nearest centroid, forming clusters

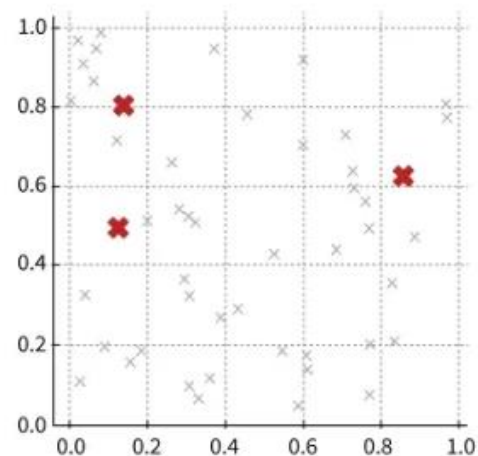
- ✱ Centroids
- ✕ Cluster 1
- ✕ Cluster 2
- ✕ Cluster 3



Choose Initial Centroids

Centroids are randomly chosen from the data points. These represent the initial cluster centers.

- ✱ Centroids
- ✕ Data Points



The algorithm will categorize the items into "k/k" groups or clusters of similarity. To calculate that similarity we will use the Euclidean distance as a measurement. The algorithm works as follows:

1. **Initialization:** We begin by randomly selecting k cluster centroids.

2. **Assignment Step:** Each data point is assigned to the nearest centroid, forming clusters.
3. **Update Step:** After the assignment, we recalculate the centroid of each cluster by averaging the points within it.
4. **Repeat:** This process repeats until the centroids no longer change or the maximum number of iterations is reached.

The goal is to partition the dataset into k clusters such that data points within each cluster are more similar to each other than to those in other clusters.

Selecting the right number of clusters is important for meaningful segmentation to do this we use Elbow Method for optimal value of k in KMeans which is a graphical tool used to determine the optimal number of clusters (k) in K-means.

Mathematical Formulation

1. Euclidean Distance

The distance between a data point x and a centroid c is calculated as:

$$d(x, c) = \sqrt{\sum_{i=1}^n (x_i - c_i)^2}$$

where:

- x_i is the i^{th} feature of the data point.
- c_i is the i^{th} feature of the centroid.
- n is the number of features.

2. Centroid Update

After assigning all data points to clusters, each centroid is updated as the mean of all points belonging to that cluster:

$$c_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

where:

- C_j is the set of points assigned to the j^{th} cluster.
- $|C_j|$ is the number of points in that cluster.

Uses of K-Means Clustering

K-Means is popular in a wide variety of applications due to its simplicity, efficiency and effectiveness. Here's why it is widely used:

1. **Data Segmentation:** One of the most common uses of K-Means is segmenting data into distinct groups. For example, businesses use K-Means to group customers based on behavior, such as purchasing patterns or website interaction.
2. **Image Compression:** K-Means can be used to reduce the complexity of images by grouping similar pixels into clusters, effectively compressing the image. This is useful for image storage and processing.
3. **Anomaly Detection:** K-Means can be applied to detect anomalies or outliers by identifying data points that do not belong to any of the clusters.
4. **Document Clustering:** In natural language processing (NLP), K-Means is used to group similar documents or articles together. It's often used in applications like recommendation systems or news categorization.
5. **Organizing Large Datasets:** When dealing with large datasets, K-Means can help in organizing the data into smaller, more manageable chunks based on similarities, improving the efficiency of data analysis.

Implementation of K-Means Clustering

We will be using blobs datasets and show how clusters are made using Python programming language.

Step 1: Importing the necessary libraries

Expectation-Maximization Algorithm - ML

The Expectation-Maximization (EM) algorithm is an iterative optimization technique used to estimate unknown parameters in probabilistic models, particularly when the data is incomplete, noisy or contains hidden (latent) variables. It works in two steps:

- **E-step (Expectation Step):** Using the current parameter estimates, the algorithm calculates the expected values of the missing or hidden variables. Essentially, it assigns probabilities or "responsibilities" to different hidden outcomes given the observed data.
- **M-step (Maximization Step):** With these updated expectations from the E-step, the algorithm then re-estimates the model parameters by maximizing the expected log-likelihood. This improves how well the model explains the observed data.



Expectation and Maximization in EM Algorithm

These two steps are repeated until convergence, which typically means that:

- The parameter values stop changing significantly or
- The log-likelihood improves only by a negligible amount.

By iteratively repeating these steps the EM algorithm seeks to maximize the likelihood of the observed data.

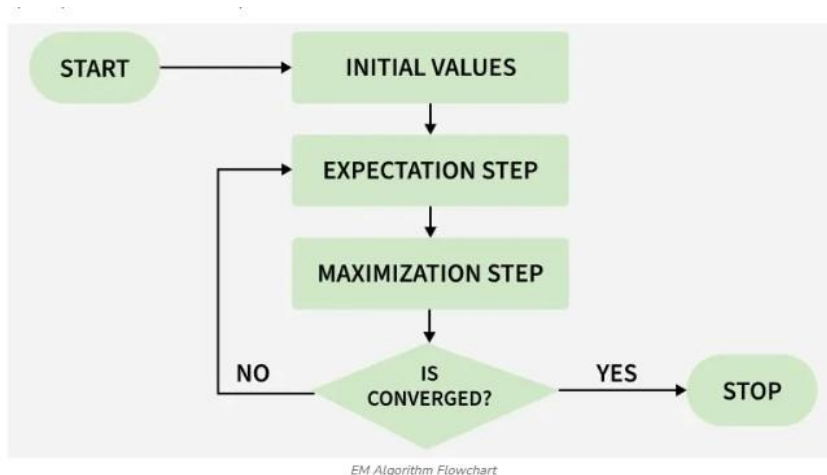
Key Concepts

Lets understand about some of the most commonly used key terms in the Expectation-Maximization (EM) Algorithm:

- **Latent Variables:** Variables that are not directly observed but are inferred from the data. They represent hidden structure (e.g., cluster assignments in Gaussian Mixture Models).
- **Likelihood:** The probability of the observed data given a set of model parameters. EM aims to find parameter values that maximize this likelihood.
- **Log-Likelihood:** The natural logarithm of the likelihood function. It simplifies calculations (turning products into sums) and is numerically more stable when dealing with very small probabilities.
- **Maximum Likelihood Estimation (MLE):** A statistical approach to estimating parameters by choosing the values that maximize the likelihood of observing the given data. EM extends MLE to cases with hidden or missing variables.
- **Posterior Probability:** In Bayesian inference, this represents the probability of parameters (or latent variables) given the observed data and prior knowledge. In EM, posterior probabilities are used in the E-step to estimate the "responsibility" of each hidden variable.
- **Convergence:** The stopping criterion for the iterative process. EM is said to converge when updates to parameters or improvements in log-likelihood become negligibly small, meaning the algorithm has reached a stable solution.

Working

Here's a step-by-step breakdown of the process:



1. Initialization: The algorithm starts with initial parameter values and assumes the observed data comes from a specific model.

2. E-Step (Expectation Step):

- Find the missing or hidden data based on the current parameters.
- Calculate the posterior probabilities (responsibilities) of the latent variables based on the observed data and current parameter estimates.

3. M-Step (Maximization Step):

- Update the model parameters by maximize the log-likelihood.
- The better the model the higher this value.

4. Convergence:

- Check if the model parameters are stable and converging.
- If the changes in log-likelihood or parameters are below a set threshold, stop. If not repeat the E-step and M-step until convergence is reached

Applications

- **Clustering:** Used in Gaussian Mixture Models (GMMs) to assign data points to clusters probabilistically.
- **Missing Data Imputation:** Helps fill in missing values in datasets by estimating them iteratively.
- **Image Processing:** Applied in image segmentation, denoising and restoration tasks where pixel classes are hidden.
- **Natural Language Processing (NLP):** Used in tasks like word alignment in machine translation and topic modeling (LDA).
- **Hidden Markov Models (HMMs):** EM's variant, the Baum-Welch algorithm, estimates transition/emission probabilities for sequence data.

Advantages

- **Monotonic improvement:** Each iteration increases (or at least never decreases) the log-likelihood.
- **Handles incomplete data well:** Works effectively even with missing or hidden variables.
- **Flexibility:** Can be applied to many probabilistic models, not just mixtures of Gaussians.
- **Easy to implement:** The E-step and M-step are conceptually simple and often have closed-form updates.

Limitations

- **Slow convergence:** Convergence can be very gradual, especially near the optimum.
- **Initialization sensitive:** Requires good initial parameter guesses; poor choices may yield bad solutions.
- **No guarantee of global best solution:** Unlike some optimization methods, EM doesn't guarantee reaching the absolute best parameters.
- **Computationally intensive:** For large datasets or complex models, repeated iterations can be costly.

Mixtures of Latent Variable Models

Latent variables are an essential concept in statistics, machine learning, and various scientific disciplines, particularly in areas involving complex data analysis and modelling. Unlike observable variables, which can be directly measured or observed, latent variables represent underlying factors or constructs that are not directly observable but are inferred from other measurable variables.

In simple terms, a latent variable is a hidden or unobservable variable that influences observed data. These variables are not directly measurable but are inferred through mathematical models that relate them to observable data. For example, consider the concept of intelligence. Intelligence cannot be directly observed or measured, but it can be inferred through measurable indicators such as performance on tests, problem-solving skills, or decision-making abilities. Intelligence, in this case, is a latent variable.

Types of Latent Variables

Latent variables can broadly be categorized into two types:

- **Continuous Latent Variables:** These are unobservable variables that take on continuous values. Examples include intelligence, socioeconomic status, or health. In machine learning, continuous latent variables often appear in probabilistic models like Gaussian Mixture Models (GMM) and Factor Analysis.
- **Categorical Latent Variables:** These are unobservable variables that represent discrete categories or groups. For instance, a person's mood (happy, sad, neutral) or personality type (introvert, extrovert) can be treated as categorical latent variables. These types of variables are often used in models like Latent Class Analysis (LCA) and Hidden Markov Models (HMMs).

Importance of Latent Variables in Modeling

Latent variables are critical in several domains as they help in simplifying complex models and capturing the underlying patterns that drive observable data. Some of the key areas where latent variables are important include:

- **Psychometrics:** In psychological testing and educational assessments, latent variables are used to measure attributes like intelligence, anxiety, motivation, and personality traits. For example, in IQ testing, latent variables represent the intelligence level that is inferred from performance on various tasks.
- **Social Sciences:** Researchers use latent variables to study complex constructs like attitudes, beliefs, and social status. Structural Equation Modeling (SEM) is a popular method in social sciences where latent variables are used to explore relationships between observed and unobserved variables.
- **Machine Learning:** In unsupervised learning models like Principal Component Analysis (PCA), latent variables capture underlying features or dimensions in high-dimensional data, such as in image or text analysis. In deep learning, latent variables play a crucial role in Autoencoders and Variational Autoencoders (VAEs), where they help represent compressed representations of input data.
- **Economics:** In economic modeling, latent variables are used to describe concepts like market sentiment or consumer preferences, which are not directly measurable but influence observable economic behavior.

Examples of Latent Variable Models

There are various statistical and machine learning models that use latent variables to uncover hidden structures in data:

- **Factor Analysis:** A technique used to describe variability among observed variables in terms of fewer unobserved (latent) variables called factors. For example, in psychology, factor analysis is used to find underlying factors influencing performance on cognitive tests.
- **Latent Dirichlet Allocation (LDA):** A widely-used topic modeling algorithm in natural language processing (NLP) that represents documents as mixtures of latent topics. These topics are inferred from the words in the document, making topics the latent variables.
- **Hidden Markov Models (HMMs):** A probabilistic model where the system being modeled is assumed to be a Markov process with hidden (latent) states. HMMs are widely used in time-series analysis, speech recognition, and bioinformatics.

- **Structural Equation Modeling (SEM):** A combination of factor analysis and regression models used to analyze structural relationships between latent and observed variables.

Benefits of Using Latent Variables

- **Simplification of Complex Models:** Latent variables allow researchers and data scientists to capture underlying structures that explain complex relationships in the data. This leads to more interpretable models and reduces the dimensionality of data.
- **Increased Model Accuracy:** By accounting for hidden factors, models that include latent variables often result in more accurate predictions or estimates.
- **Theoretical Insight:** Latent variables offer insights into abstract constructs and theories that cannot be directly measured, providing a deeper understanding of human behavior, societal trends, or hidden patterns in data.

Challenges with Latent Variables

While latent variables are incredibly useful, there are also challenges associated with their use:

- **Model Identification:** Estimating latent variables can be difficult since they are not directly observable. This requires complex mathematical models and assumptions, which may not always hold true.
- **Interpretation:** In some cases, latent variables can be difficult to interpret or explain, especially in high-dimensional models where the meaning of the latent variable may not be clear.
- **Overfitting:** Introducing too many latent variables in a model can lead to overfitting, where the model becomes too tailored to the training data and performs poorly on new, unseen data.

Supervised Learning After Clustering

Supervised Learning After Clustering

Clustering works with **unlabeled data** and attempts to discover **natural groups or structures** within the data.

For example, a customer dataset may be divided into:

- **Cluster 1 – High-value customers**
- **Cluster 2 – Occasional customers**
- **Cluster 3 – Low-value customers**

However, clustering only identifies groups of observations that are similar according to the selected clustering algorithm and features. It does **not necessarily provide predefined or ground-truth target labels**.

Once meaningful clusters have been discovered, the clustering results can be used as part of a supervised learning process.

This creates a connection between:

Unsupervised Learning → Supervised Learning

Why Use Supervised Learning After Clustering?

Clustering is often useful during the **exploratory stage** of a machine learning problem.

Suppose an organization has customer data but does not have predefined customer categories.

The organization can first apply clustering to discover customer segments.

Clustering Process

Customer Data

↓

Clustering Algorithm

↓

Discovered Customer Groups

- Cluster 1
- Cluster 2
- Cluster 3

After identifying and interpreting these groups, the organization may want to classify **new customers** into one of the previously discovered groups.

For example:

A new customer joins today. Which customer segment does this customer belong to?

This is a **prediction problem**.

Instead of performing the entire clustering analysis repeatedly, a supervised classifier can be trained to learn the relationship between the customer features and the previously discovered cluster assignments.

Cluster-Then-Classify Approach

The **Cluster-Then-Classify** approach consists of two major learning stages.

Stage 1 – Clustering

First, an unsupervised clustering algorithm is applied to historical data.

$X \rightarrow \text{Clusters}$

For example:

$X \rightarrow \{C1, C2, C3\}$

where:

- XX = input features
- $C1, C2, C3$ = discovered clusters

For example:

- $C1 \rightarrow$ High-value customers
- $C2 \rightarrow$ Occasional customers
- $C3 \rightarrow$ Low-value customers

Stage 2 – Create Pseudo-Labels

After clustering, every observation receives a **cluster assignment**.

For example:

Customer Income Spending Cluster

C1	80,000	90	1
C2	25,000	20	3
C3	50,000	50	2

The cluster assignment can be treated as a **pseudo-label** or **derived label** for the next learning stage.

ClusterID = {1, 2, 3}

Important:

Cluster ID is not necessarily a true class label

It is generated by the clustering algorithm.

Stage 3 – Train a Supervised Classifier

The original features are now used as inputs, while the cluster assignments are used as the target for classification.

$X \rightarrow \text{ClusterID}$

For example:

Income + Spending $\rightarrow$ ClusterID

A supervised algorithm such as a:

- Decision Tree
- Random Forest
- SVM
- k-NN

can be trained to learn this relationship.

Stage 4 – Predict the Cluster of a New Observation

When a new customer arrives, the trained classifier can predict the customer's previously defined segment.

For a new customer:

$X_{\text{new}} \rightarrow C^{\wedge}$

where:

$C^{\wedge}$ = Predicted Cluster

For example:

New Customer

- Income = ₹75,000

- Spending Score = 82

↓

Supervised Classifier

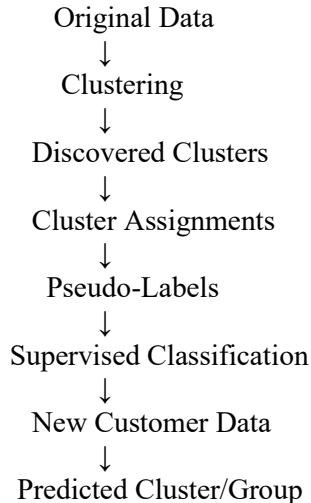
↓

Predicted Cluster = 1

↓

High-value Customer Segment

Complete Process



Spectral Clustering

Spectral Clustering is an unsupervised learning technique used to group data points based on their similarity. It uses concepts from graph theory and linear algebra to transform data into a space where clustering becomes easier.

- Works well for non-linear and complex shapes
- Uses concepts from linear algebra (eigenvalues & eigenvectors)
- Converts data into a graph structure (nodes and edges)

Types of Spectral Clustering

- **Unnormalized Spectral Clustering:** Uses the similarity values directly to group data points without any adjustment but it may not perform well when the data is uneven.
- **Normalized Spectral Clustering:** Adjusts the similarity values before clustering, which helps in getting better and more balanced clusters in most real-world datasets.
- **Random Walk Spectral Clustering:** Groups data points based on how strongly they are connected to each other in the graph.

Key Concepts

It explains how data is represented and transformed so that clusters can be identified more easily. The main concepts involved are:

1. Similarity Matrix (W)

The matrix which represents how similar two data points are. It measures the relationship between every pair of points based on distance.

$$W_{ij} = e^{-\frac{\|x_i - x_j\|^2}{2\sigma^2}}$$

- If two points are close then value is high
- If far then value is low

This forms a graph where points are nodes and similarity is edges.

2. Degree Matrix (D)

Degree matrix represents the total connection strength of each data point. It is formed by summing all similarity values for a given point.

$$D_{ii} = \sum_j W_{ij}$$

- Diagonal matrix
- Shows how strongly each point is connected

3. Graph Laplacian (L)

This matrix identifies the structure of the graph. It is obtained by subtracting the similarity matrix from the degree matrix.

$$L = D - W$$

- Core matrix in Spectral Clustering
- Helps identify connected groups

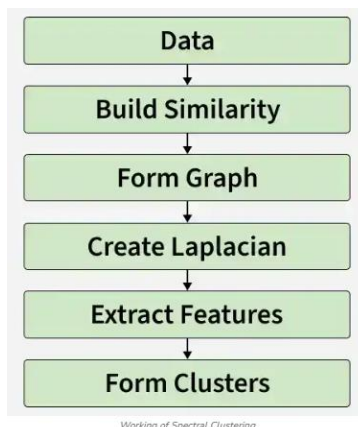
4. Eigenvalues & Eigenvectors

These are mathematical tools used to extract important patterns from the data. They help transform the data into a new space where clusters become clearer.

- Used for dimensionality reduction
- Makes clusters easier to separate

Working

The steps involved in Spectral Clustering are as follows:



Step 1: Converting data into a graph structure where

- Each point is a node
- Similar points are connected (strong connection = similar points)

Step 2: Now, Instead of looking at raw data, we study this network structure.

Step 3: The Laplacian matrix combines similarity and connection information to reveal the structure of the data.

- Strong connections (same cluster)
- Weak connections (different clusters)

Step 4: Eigenvectors are then used to rearrange the data into a new form.

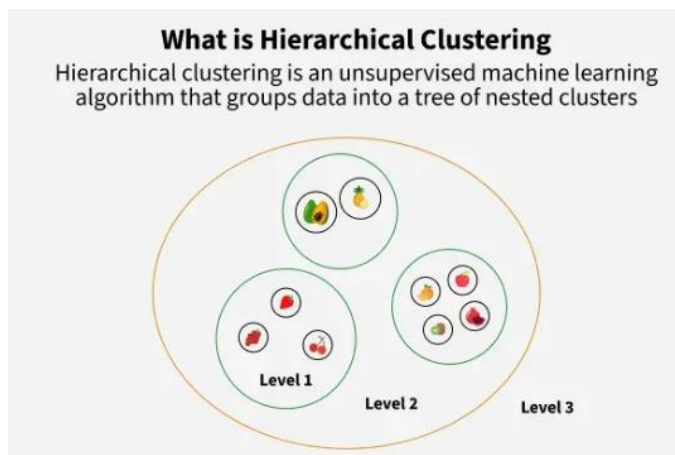
Step 5: Here, data is transformed into a new lower-dimensional space where points from same cluster move closer

Step 6: We apply K - Means algorithm where clusters become easy to detect.

Hierarchical Clustering in Machine Learning

Hierarchical Clustering is an unsupervised learning technique that groups data into a hierarchy of clusters based on similarity. It builds a tree-like structure (dendrogram) that helps visualize relationships and decide the optimal number of clusters.

- Does not require pre-selecting the number of clusters.
- Uses agglomerative or divisive approaches.
- Commonly applied in data exploration and pattern discovery.
- It is commonly used in pattern recognition, customer segmentation and image grouping.



Imagine we have four fruits with different weights: an apple (100g), a banana (120g), a cherry (50g) and a grape (30g). Hierarchical clustering starts by treating each fruit as its own group.

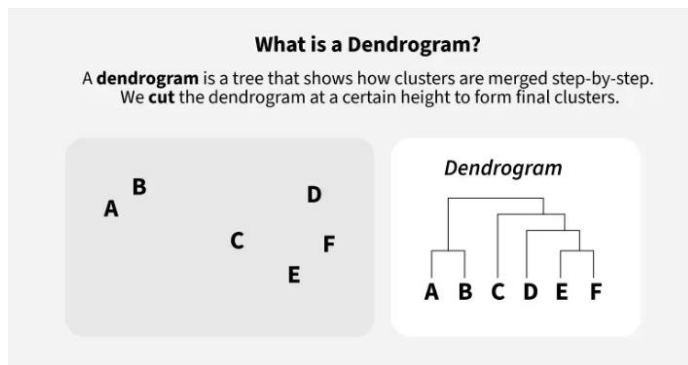
- Start with each fruit as its own cluster.
- Merge the closest items: grape (30g) and cherry (50g) are grouped first.
- Next, apple (100g) and banana (120g) are grouped.
- Finally, these two clusters merge into one.

Finally all the fruits are merged into one large group, showing how hierarchical clustering progressively combines the most similar data points.

Dendrogram

A dendrogram is like a family tree for clusters. It shows how individual data points or groups of data merge together. The bottom shows each data point as its own group and as we move up, similar groups are combined.

The lower the merge point, the more similar the groups are. It helps us see how things are grouped step by step.



Dendrogram

- At the bottom of the dendrogram the points P, Q, R, S and T are all separate.
- As we move up, the closest points are merged into a single group.
- The lines connecting the points show how they are progressively merged based on similarity.
- The height at which they are connected shows how similar the points are to each other; the shorter the line the more similar they are

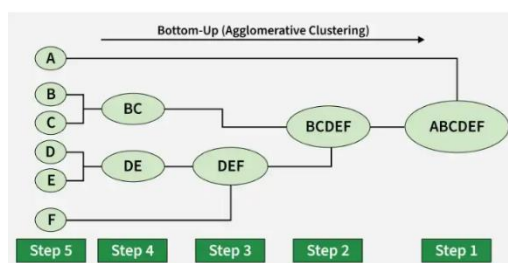
Types of Hierarchical Clustering

Now we understand the basics of hierarchical clustering. There are two main types of hierarchical clustering.

1. Agglomerative Clustering
2. Divisive clustering

1. Hierarchical Agglomerative Clustering

It is also known as the bottom-up approach or hierarchical agglomerative clustering (HAC). Bottom-up algorithms treat each data as a singleton cluster at the outset and then successively agglomerate pairs of clusters until all clusters have been merged into a single cluster that contains all data.



Workflow for Hierarchical Agglomerative clustering

1. **Start with individual points:** Each data point is its own cluster. For example if we have 5 data points we start with 5 clusters each containing just one data point.

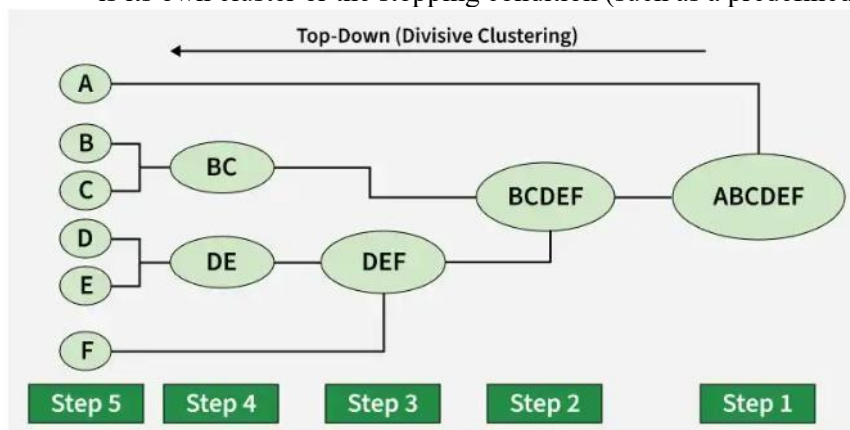
2. **Calculate distances between clusters:** Calculate the distance between every pair of clusters. Initially since each cluster has one point this is the distance between the two data points.
3. **Merge the closest clusters:** Identify the two clusters with the smallest distance and merge them into a single cluster.
4. **Update distance matrix:** After merging we now have one less cluster. Recalculate the distances between the new cluster and the remaining clusters.
5. **Repeat steps 3 and 4:** Keep merging the closest clusters and updating the distance matrix until we have only one cluster left.
6. **Create a dendrogram:** As the process continues we can visualize the merging of clusters using a tree-like diagram called a dendrogram. It shows the hierarchy of how clusters are merged.

2. Hierarchical Divisive clustering

Divisive clustering is also known as a top-down approach. Top-down clustering requires a method for splitting a cluster that contains the whole data and proceeds by splitting clusters recursively until individual data have been split into singleton clusters.

Workflow for Hierarchical Divisive clustering

1. **Start with all data points in one cluster:** Treat the entire dataset as a single large cluster.
2. **Split the cluster:** Divide the cluster into two smaller clusters. The division is typically done by finding the two most dissimilar points in the cluster and using them to separate the data into two parts.
3. **Repeat the process:** For each of the new clusters, repeat the splitting process: Choose the cluster with the most dissimilar points and split it again into two smaller clusters.
4. **Stop when each data point is in its own cluster:** Continue this process until every data point is its own cluster or the stopping condition (such as a predefined number of clusters) is met.



Computing Distance Matrix

While merging two clusters we check the distance between two every pair of clusters and merge the pair with the least distance/most similarity. But the question is how is that distance determined. There are different ways of defining Inter Cluster distance/similarity. Some of them are:

- **Min Distance:** Find the minimum distance between any two points of the cluster.
- **Max Distance:** Find the maximum distance between any two points of the cluster.
- **Group Average:** Find the average distance between every two points of the clusters.
- **Ward's Method:** The similarity of two clusters is based on the increase in squared error when two clusters are merged.



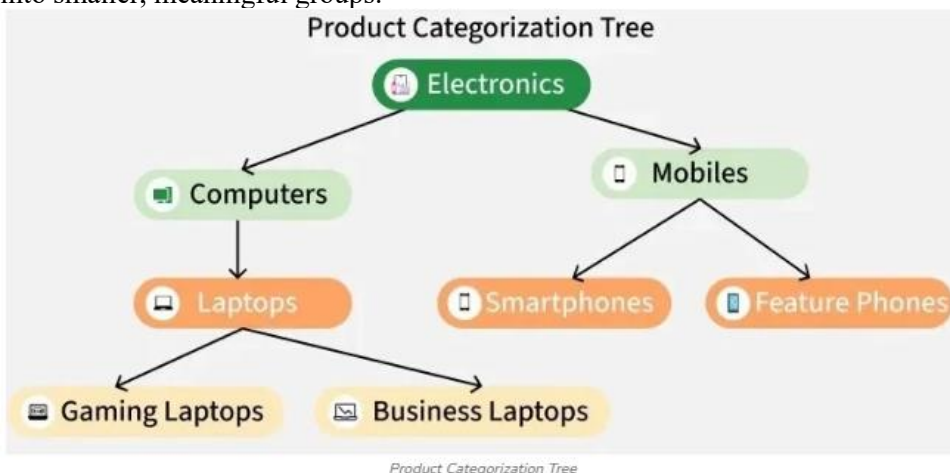
Distance Matrix

The image compares cluster distance methods:

- Min uses the shortest distance between clusters
- Max uses the longest
- Group Average computes the mean of all pairwise distances
- Ward's method minimizes the increase in within-cluster variance during merging

Divisive Clustering

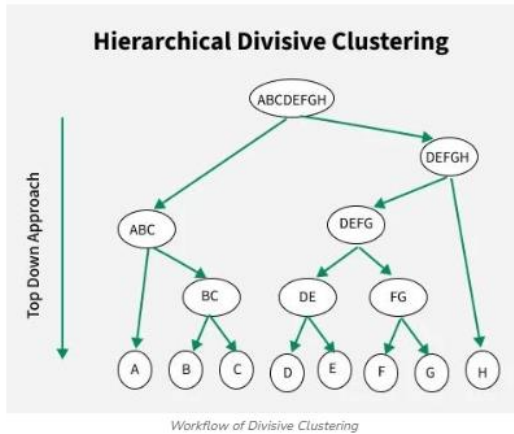
Divisive Clustering is a type of hierarchical clustering that follows a top-down approach. It starts by placing all data points into one large cluster and then recursively splits that cluster into smaller ones based on differences or distances between the points. This process continues until each cluster contains only similar data points or meets a stopping condition. It is the opposite of agglomerative clustering, which builds clusters from the bottom up. It is useful when we want to break down a broad category into smaller, meaningful groups.



Product Categorization Tree

Workflow of Divisive Clustering

1. **Start with all data points:** Begin with one big group ABCDEFGH.
2. **First split:** Divide it into two groups ABC and DEFGH.
3. **Split again:** The group ABC is divided into A and BC while the group DEFGH is split into DEFG and H.
4. **Keep dividing:** We continue dividing these new groups. BC is split into B and C, DEFG is divided into DE and so on. At this stage most of the data points are now in their individual groups.
5. **Finish:** Stop when all points are separated.



Real-World Applications of Divisive Clustering

1. **Biological Taxonomy:** Divisive clustering is used in biology to sort living things from broad to specific categories. For example, start with "Animals", then split into "Mammals", then into "Primates" and finally into specific species like "Humans".
2. **Product Categorization in E-commerce:** Online shopping websites organize products using a top-down category structure.
3. **Document and Topic Classification:** Large libraries or article databases use it to arrange documents into main topics and detailed subtopics. Like articles on "Science" can be divided into "Physics", then into "Quantum Mechanics" and so on.
4. **Customer Segmentation in Marketing:** Companies use it to group customers based on their characteristics or buying behavior. For example, all customers are grouped initially, then split into "Frequent Buyers", "New Users" and "High Spenders".
5. **Medical Diagnosis Systems:** Doctors or AI systems use this method to narrow down possible diseases from broad symptom categories. Like from the symptom "fever", it may be split into "viral", then "flu" and then into specific flu types.